

Lab class 3 Exercise Sheet

Model answers

Question 1:

On a construction site, some stakes have been hammered into the ground. As you watch, a worker ties a rope to one of the stakes and then runs it around another stake, some distance away, pulling it tight. When you ask the worker what he's doing, he says, 'See, I need to run a cable from stake A over there, via stake B, here, to stake C all the way over there. So I tie this rope to A, wind it around B and then tie the end to C. This way I know how long a cable I need to buy.' This makes you wonder: When is it possible to measure the sum of the pairwise distances between all pairs of stakes, by using a single rope in this way?

To measure the sum of all pairwise distances between all stakes, we need to string the rope in such a pattern that it traces all pairwise distances once. Let's consider the graph where the stakes are nodes and a link indicates that the rope connects the respective pair of stakes. To measure the sum of the pairwise distances, we need to create a fully connected graph. We can trace a given path with a single rope if the graph has an *Eulerian path* (see 3rd lecture). We know from Euler's bridge theorem that such a path exists if there are at most two nodes of odd degree. In a fully connected network, each node has the same degree, and the degrees are odd when the number of nodes is even, and vice versa. So the answer is, we can measure the sum of all pairwise distances between stakes if either the number of stakes is less than three or if it is odd.

Question 2:

In this exercise, I want you to pick two networks from one of the repositories mentioned in the first lecture. Make sure that these networks are from different domains:

- social network (e.g., Facebook, Twitter, or collaboration networks)
- biological network (e.g., Protein-protein interactions)
- technological network (e.g., Internet topology, power grid)

Pick networks with roughly the same number of nodes. Preserving density here might not be particularly helpful so do feel free to pick networks of interest even if they have a very different number of edges for the same number of nodes.

Download those two networks then write code to compute and visualize their k-core structure. What do you see and what does it tell you about those networks?

Next, write code to iteratively ‘attack’ the network such that at each iteration you delete a node, either picking it randomly or picking it according to some metric of your choice (e.g., degree is a common one but you could pick another one, e.g., betweenness centrality). At each iteration (including before the first removal), calculate the size of the largest connected component.

Plot the curve that shows the size of the largest connected component against the size of the network. Compare the curves when picking nodes at random vs picking nodes according to your metric. What does it tell you about the network? And how does that change depending on the network?

Here, I’ve picked two networks of roughly the same size (in number of nodes). One is an infrastructure network, [Euroroads](#) and one is a co-authorship network, [Network Science](#). The former is a road network located mostly in Europe. The network is undirected; nodes represent cities and an edge between two nodes denotes that they are connected by an E-road. The latter is a network of co-authorships in the area of network science. These are both edge lists but note that they have a header with a couple of comments preceded by a percentage sign so to load it we need to make sure to specify the comments argument in the `read_edgelist` function call.

Let’s first look at k-core decomposition.

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
import random

def k_core_decomposition(G):
    core_numbers = nx.core_number(G)
    max_k = max(core_numbers.values())
    highest_k_core = G.subgraph([
        node for node in G.nodes if core_numbers[node] == max_k
    ])

    # Summary statistics for the highest k-core
    num_nodes = highest_k_core.number_of_nodes()
    num_edges = highest_k_core.number_of_edges()
```

```

    avg_degree = (sum(dict(highest_k_core.degree()).values()) / num_nodes
                  if num_nodes > 0 else 0)
    density = nx.density(highest_k_core)

    print(f'Highest k-core (k={max_k}) statistics:')
    print(f'Number of nodes: {num_nodes}')
    print(f'Number of edges: {num_edges}')
    print(f'Average degree: {avg_degree:.2f}')
    print(f'Density: {density:.4f}')

    # Plot k-core decomposition
    plt.figure(figsize=(8, 6))
    for k in sorted(set(core_numbers.values())):
        subgraph = G.subgraph([
            node for node in G.nodes if core_numbers[node] >= k
        ])
        pos = nx.spring_layout(subgraph)
        nx.draw(subgraph, pos, node_size=50,
                with_labels=False, label=f'k-core {k}')
        plt.title(f'k-core {k}')
        plt.show()

    return core_numbers

euroroad = nx.read_edgelist(
    './data/out.subelj_euroroad_euroroad', comments='#'
)
netscience = nx.read_edgelist(
    './data/out.dimacs10-netscience', comments='#'
)

k_core_net1 = k_core_decomposition(euroroad)
k_core_net2 = k_core_decomposition(netscience)

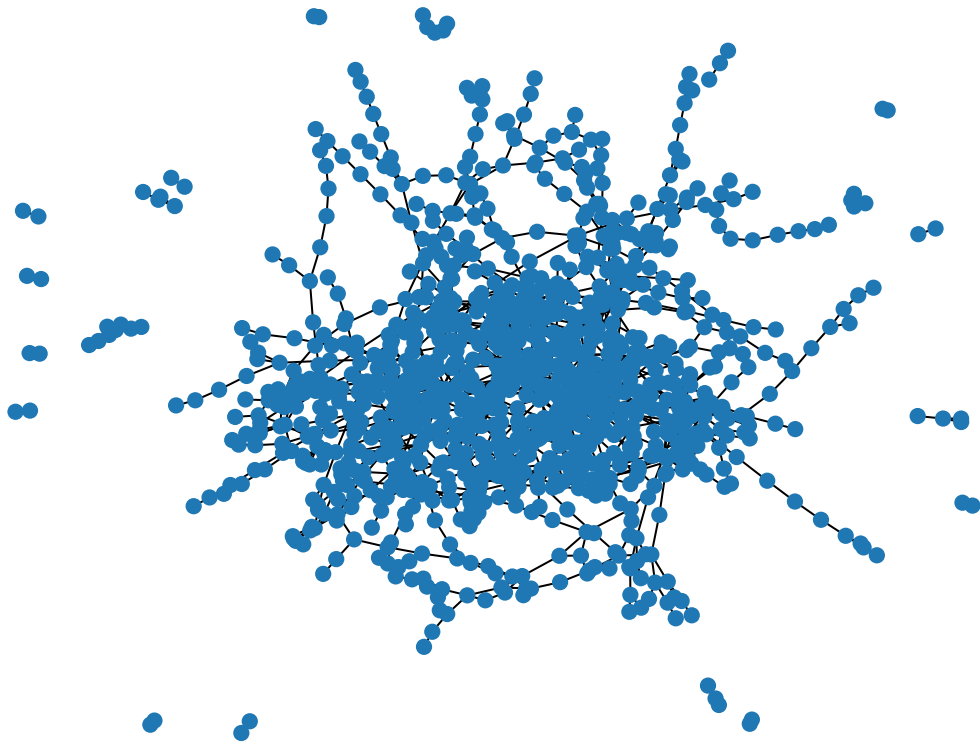
```

```

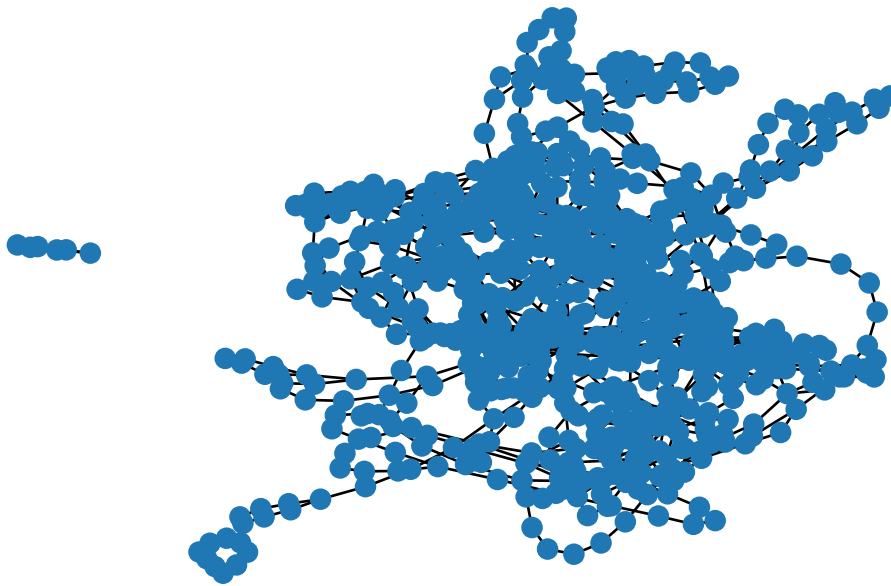
Highest k-core (k=2) statistics:
Number of nodes: 751
Number of edges: 1017
Average degree: 2.71
Density: 0.0036

```

k-core 1



k-core 2



Highest k-core (k=19) statistics:

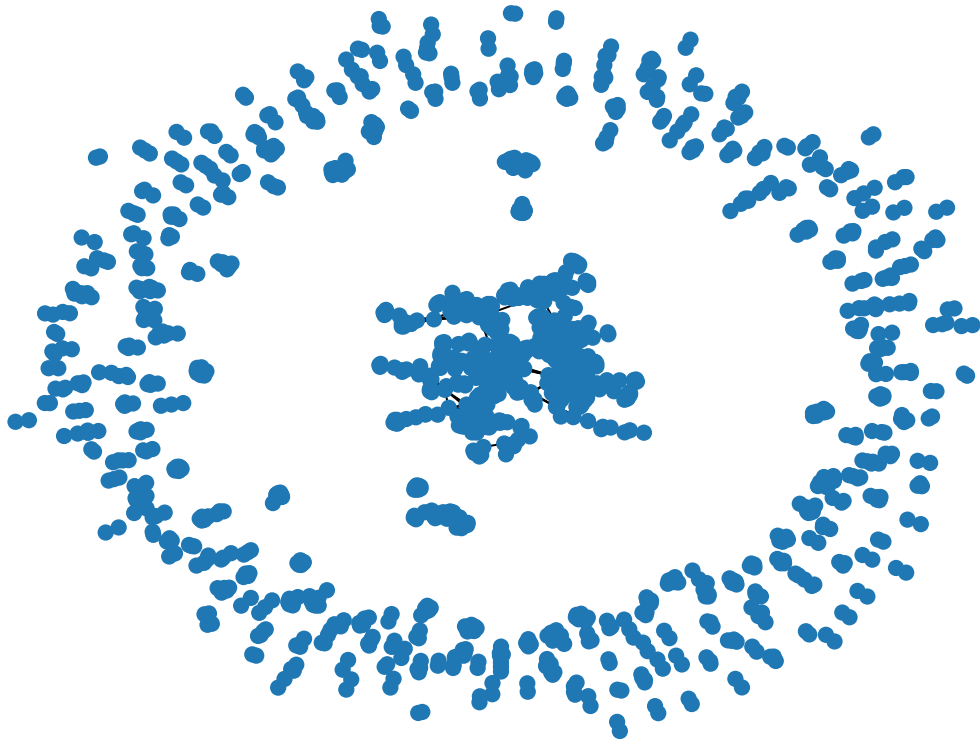
Number of nodes: 20

Number of edges: 190

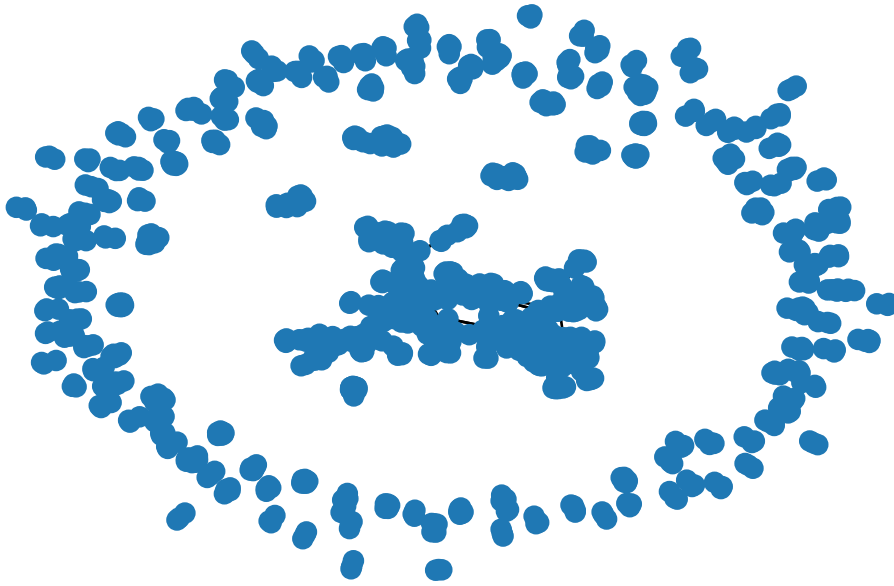
Average degree: 19.00

Density: 1.0000

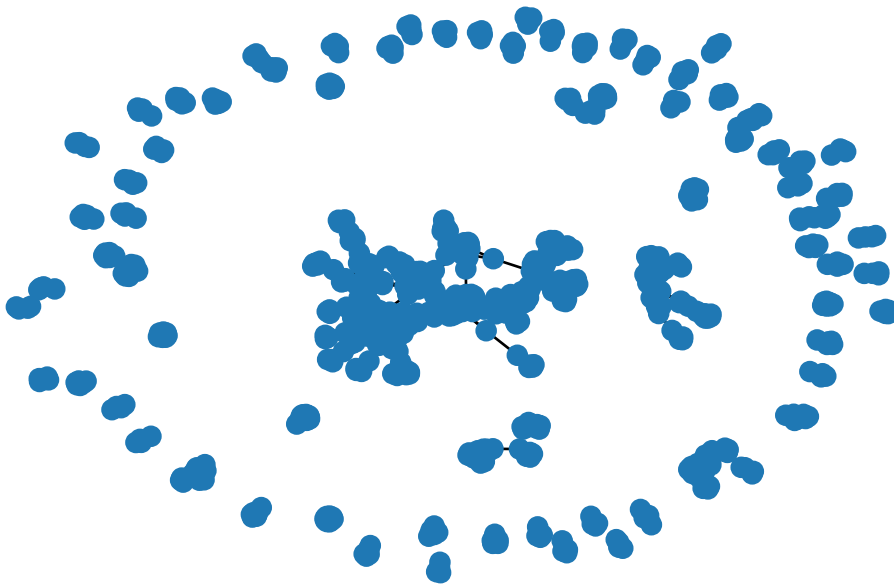
k-core 1



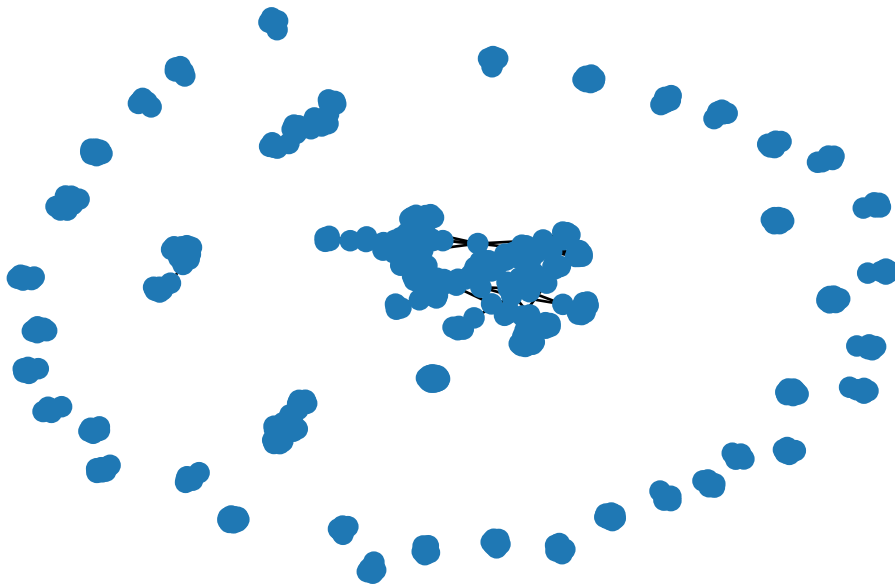
k-core 2



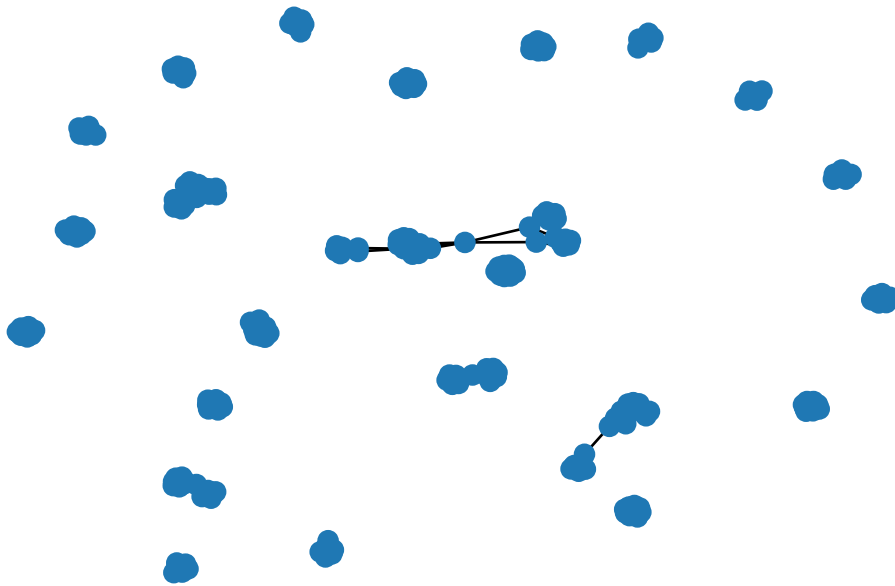
k-core 3



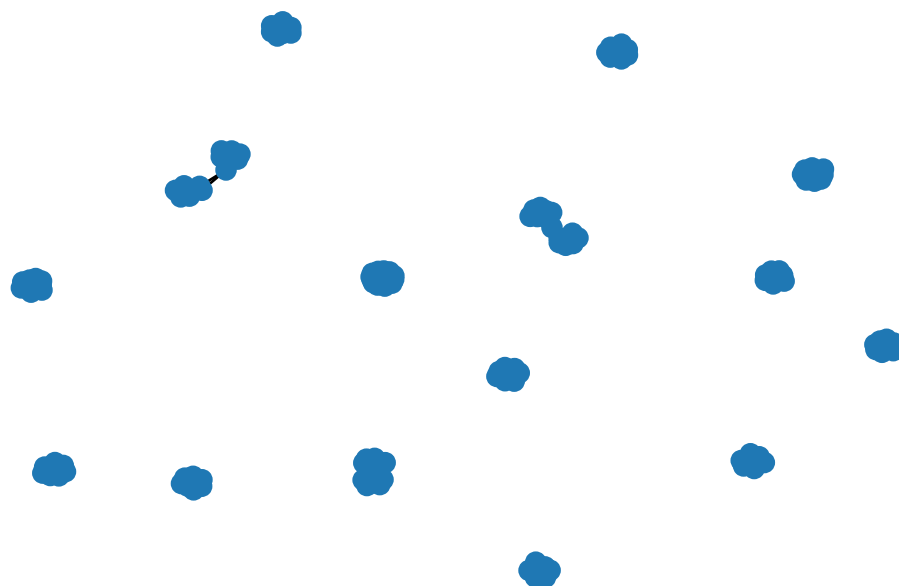
k-core 4



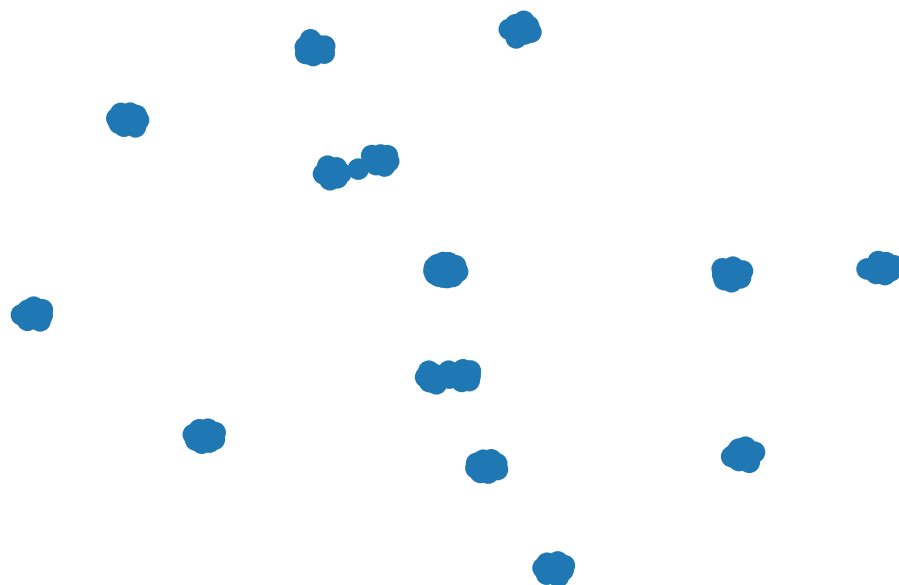
k-core 5



k-core 6



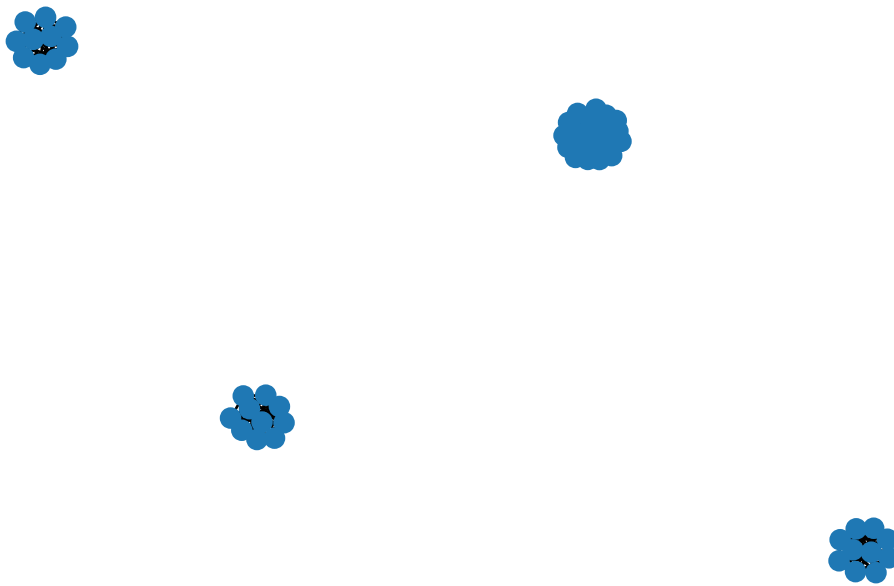
k-core 7



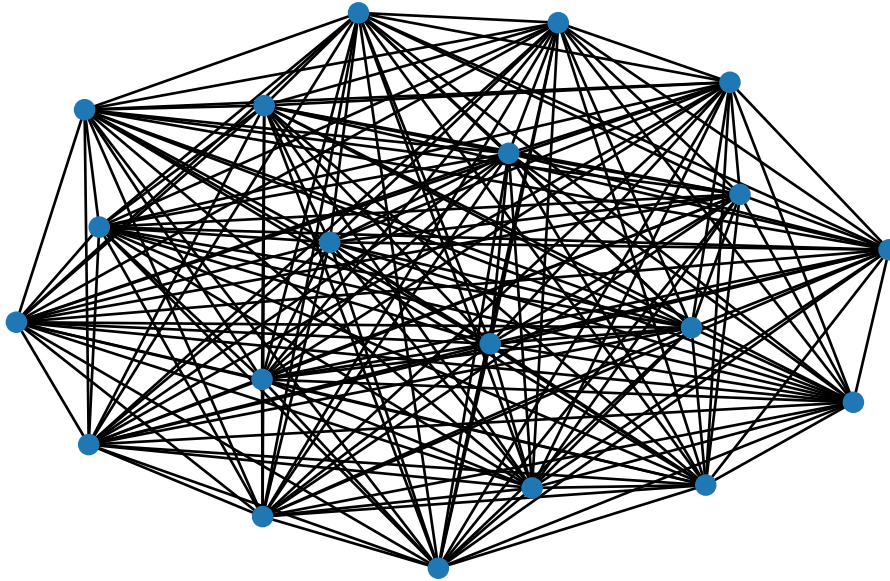
k-core 8



k-core 9



k-core 19



Clearly, we are getting two very distinct ‘behaviours’. With the road network, the highest k-core is 2, i.e., every node has at least 2 connections. This makes sense as road networks typically form planar graphs with lots of degree-2 nodes. Here, it suggests that nodes are in a sparse grid or tree-like structure (which you can see, in fact). There are no strongly interconnected subgraphs. Clustering is weak. Even if cities have high-degree, they do not form a fully interconnected “rich club”. With the citation network, we see the emergence of a number of separate highly-clustered subgraphs (up to $k = 10$), culminating with a single very dense, clustered subgraph ($k = 19$). This suggests a network with a hierarchical structure of influence and connectivity. The k-core of 19 is likely a set of highly influential papers that frequently cite each other. You could think of them as the backbone of the field. You could also think of it as evidence for the presence of a clique... The separate clusters up to $k = 10$ are evidence of fragmentation which may indicate that the field is diverse with multiple distinct subfields or groups of thoughts. These lower-k cores could be groups of papers that are well-cited but not yet densely interconnected with the highest-impact research.

Let’s now turn to network robustness. The code below provides for random-, degree- and betweenness-based attacks. I will only show results using random- and degree-based. Do play with the code to see any difference when using betweenness-based attack. There are a few ways to do this BTW. I think it’s helpful to plot fractions rather than absolute numbers for ease of comparison.

```

def largest_connected_component_size(G):
    if len(G) == 0:
        return 0
    return max(len(c) for c in nx.connected_components(G))

def attack_network(G, strategy='random'):
    G = G.copy()
    original_size = G.number_of_nodes()
    sizes = [largest_connected_component_size(G) / original_size]
    removed_fractions = [0] # Fraction of removed nodes

    while G.number_of_nodes() > 1:
        if strategy == 'random':
            node_to_remove = random.choice(list(G.nodes))
        elif strategy == 'degree':
            node_to_remove = max(G.degree, key=lambda x: x[1])[0]
        elif strategy == 'betweenness':
            betweenness = nx.betweenness_centrality(G)
            node_to_remove = max(betweenness, key=betweenness.get)
        else:
            raise ValueError(
                "Invalid strategy. Choose 'random', 'degree', or 'betweenness'."
            )

        G.remove_node(node_to_remove)
        sizes.append(largest_connected_component_size(G) / original_size)
        removed_fractions.append(
            (original_size - G.number_of_nodes()) / original_size
        )

    return removed_fractions, sizes

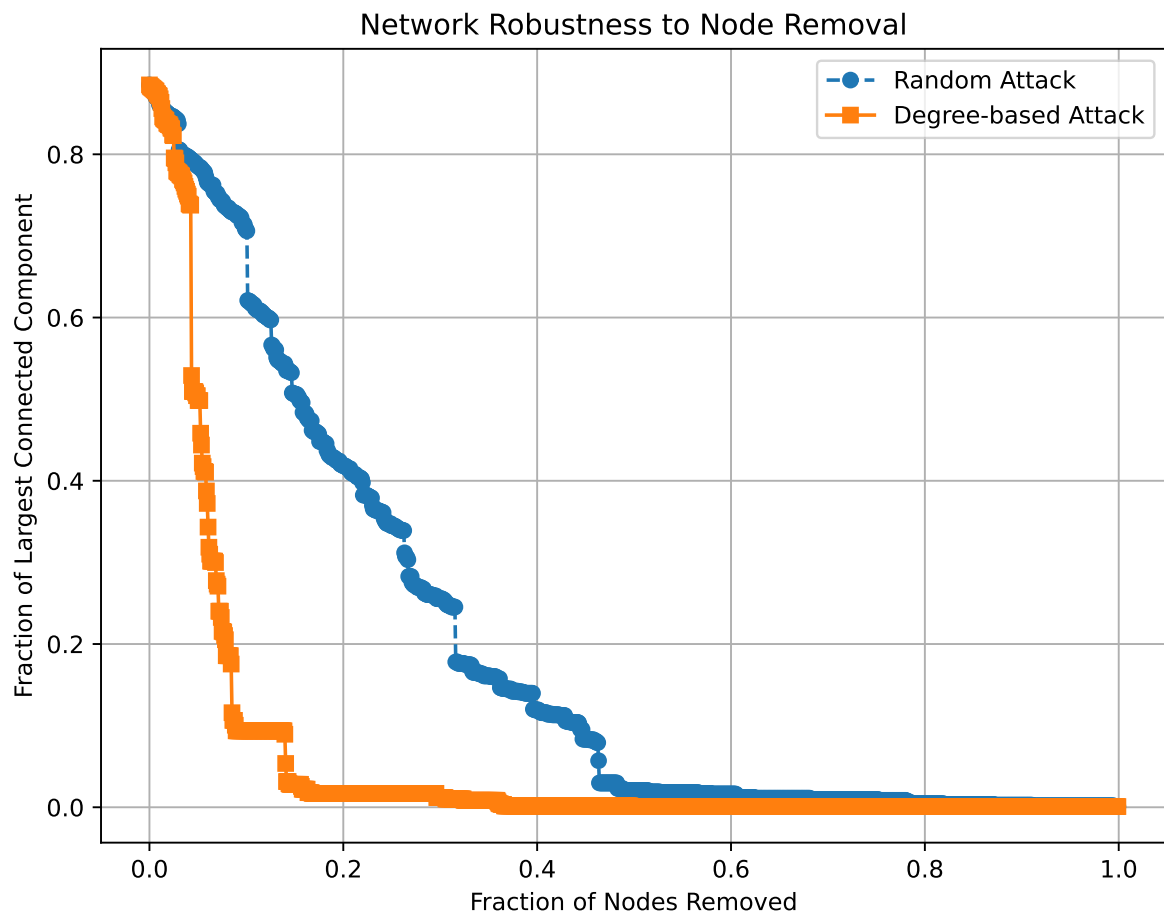
def plot_attack_results(G):
    removed_random, lcc_random = attack_network(G, strategy='random')
    removed_degree, lcc_degree = attack_network(G, strategy='degree')

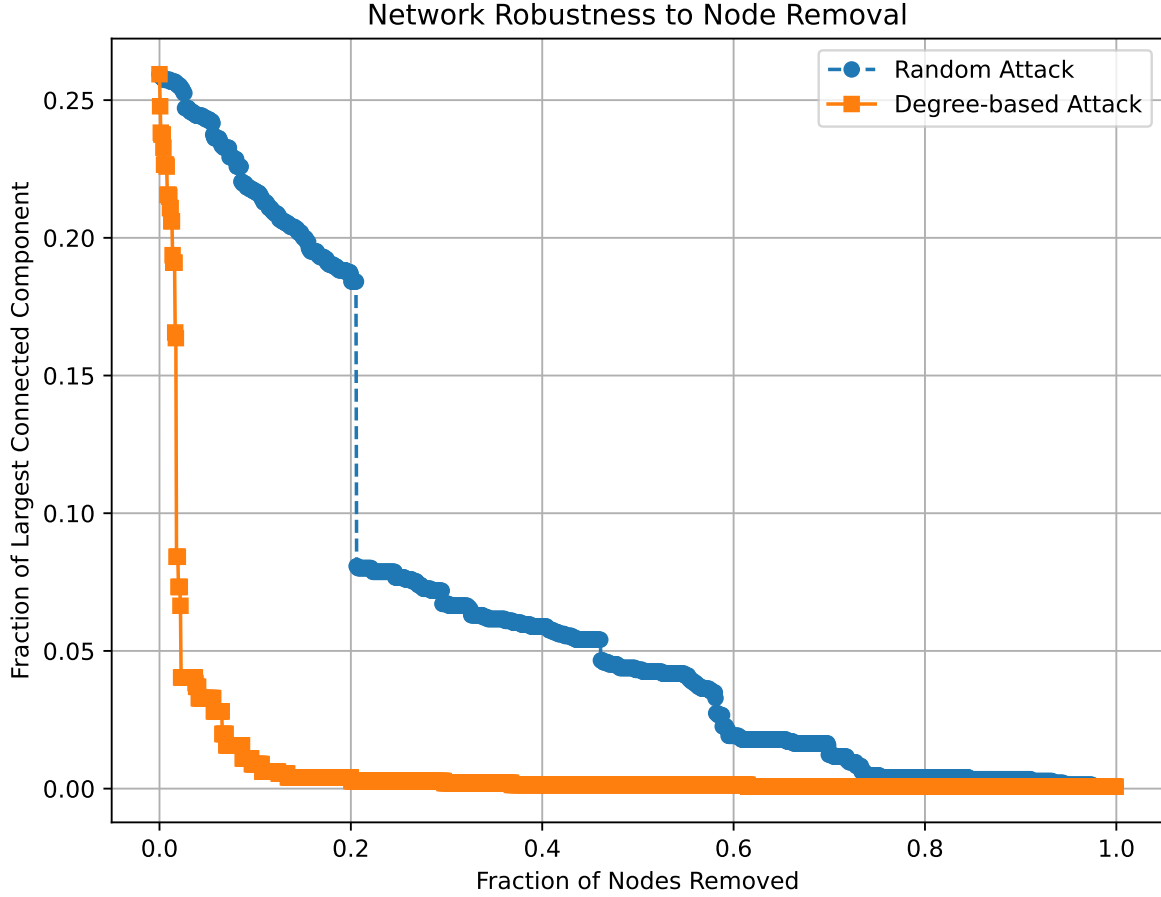
    plt.figure(figsize=(8, 6))
    plt.plot(removed_random, lcc_random,
             label='Random Attack', linestyle='--', marker='o')
    plt.plot(removed_degree, lcc_degree,
             label='Degree-based Attack', linestyle='-', marker='s')
    plt.xlabel('Fraction of Nodes Removed')

```

```
plt.ylabel('Fraction of Largest Connected Component')
plt.title('Network Robustness to Node Removal')
plt.legend()
plt.grid()
plt.show()
```

```
plot_attack_results(euroroad)
plot_attack_results(netscience)
```





So here again, there are some notable differences both between and within networks. For euroroads, the network is certainly very susceptible to the removal of some high-degree nodes but, for the difference between random- and degree-based attacks is not particularly strong. This makes sense if one considers what we discussed previously, i.e., the fact that the road network is like a grid. There isn't such a difference between a randomly picked node and a 'targeted' node. In contrast, the citation network shows a very strong difference between targeted and random attacks. This reflects the hierarchical connectivity we mentioned before. Once the rich-club is 'removed' (the almost vertical drop at the start), the citation network is strongly fragmented. Picking nodes at random, however, is unlikely to disconnect the network and we see a very gradual reduction of the size of the largest connected component. Do note, however, that from the start, the largest connected component is a much smaller fraction of the total network than in the case of the road network. Now why would that happen? A possibility is that of *temporal effect*. In evolving networks (like citations or collaboration networks), early-stage networks tend to be fragmented before they grow into a strongly connected system. The dataset we use doesn't provide extra information such as names/dates but an interesting idea, were that data available (and it is... in other datasets) would be to 'track' the evolution of the

groupings, e.g., when/how do the separate clusters end up merging. The road network, on the other hand, shows a large well-connected giant component (over 80% of the network).